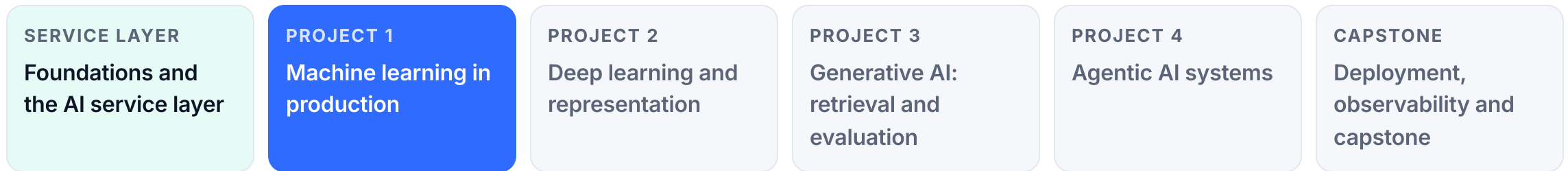


Machine learning in production

AI ENGINEERING: GENAI AND AGENTIC SYSTEMS

Today you build Project 1, `nw/triage`: a calibrated priority classifier that reaches production only through a gate, a pipeline, a registry and a person's approval. Then the service that serves it on your track, measures drift on every request, shadows the next version, and a retraining job that never promotes on its own.

The service layer gave every model call one door; Project 1 needs none



Every part asks which kind of AI fits the job. For ordering Northwind's queue by priority, the answer is a linear model on word and character features, trained in seconds. The agent calls it as a tool; the capstone puts it in front of the agent as a cheap pre-filter and promotes it behind a canary.

Today you build a triage model that only a person can promote

- 1 Read the data and the contract
- 2 The naive model meets the gate
- 3 Class weights
- 4 A threshold, not argmax
- 5 Calibrate the probabilities
- 6 Track and compare runs
- 7 Train through the pipeline
- 8 Read the model card and the promotion log
- 9 Read the registry
- 10 Approve the candidate and see it served
- 11 Alive is not ready; drift
- 12 Shadow the next version
- 13 The platform's monitor
- 14 The container and the packaging
- 15 The retraining loop, paused
- 16 Check the part and commit

A pipeline can propose. Only a person promotes.

A small model beats an LLM when the task is narrow and the volume is high

Latency

Topic classification: a fine-tuned encoder took about 0.02 s per text; most LLMs 0.6 to 1.4 s; GPT-5 5.5 s.

Control

Fine-tuned BERT-style models beat zero-shot GPT-4 and Claude Opus on every classification task tested.

Cost

No tokens: a prediction is CPU time on a service you already run. Project 1 answers in 1.7 ms at the median.

Audit

US lenders must give specific reasons for a credit denial: "there is no special exemption for artificial intelligence".

MLOps is a maturity ladder, and most teams stand on its bottom rung

Level	What it means	In Project 1
0	manual, script-driven, infrequent releases	where a notebook and a copied model file stop
1	an automated pipeline, continuous training, data and model validation	the data contract, the pipeline, the gate, the registry, the paused retraining job
2	automated CI/CD for the pipeline itself	not claimed: the pull-request gate and the capstone's delivery are pieces toward it

"MLOps level 0 is common in many businesses." Google Cloud Architecture Center

Only a small fraction of a real-world ML system is the ML code. Today writes that fraction first, then the rest of the system around it.

Accuracy is a trap on four percent P0

3.9%

of Northwind's tickets are P0. A model that never predicts P0 is 96 percent accurate on that class and useless

Metric	Why it matters here
Accuracy	dominated by the common classes; hides P0
Macro - F1	rewards getting the rare class right
P0 recall	the commercial number: a missed P0 is an SLA breach
P0 precision	the cost of the recall you buy
Brier and ECE	whether a threshold on the probability means anything

Nothing trains until the data passes its contract

Expectation	What it catches
Every row matches the pydantic contract	a renamed column, an empty body
P0 between 2 and 8 percent	an export that drops or floods the rare class
Validation and test at least 8 percent	numbers that rest on too few rows
Duplicates under 0.5 percent, no split leak	a test number that is a lie
Every label in every split	a sweep that cannot run

`make data-check`

rows 7950, splits 6361 / 795 / 794, P0 3.9%, de 4.9%, text length deciles from 401, duplicates 0, **DATA OK.**

The same run writes the data profile. It ships inside every artifact, and the pipeline's first step runs the same check.

The naive model sends 13 real emergencies to the ordinary queue

18 of 31

true P0 tickets caught at a fixed 0.5 on the test split. The P0 row reads 18, 13, 0, 0

- Accuracy 0.694, macro-F1 0.688 against a majority baseline of 0.414 and 0.146
- P0 precision 0.900: usually right when it says P0, the wrong trade for an outage
- **GATE FAILED**: P0 recall 0.581 is below the bar 0.85, and the candidate misses 10 more P0 tickets than production; the bar allows 1

The same refusal the pre-work pipeline printed at its register step. A model can pass most of a test suite and still be useless for the one thing the business needs.

Thirty-one emergencies make a wide interval, so the gate counts tickets

28 of 31

production's P0 recall on the test split: 0.903, with a 95 percent Wilson interval of 0.751 to 0.967

- A **Wilson interval** is the range of true recall values the 31 tickets are consistent with; it stays inside 0 to 1 and behaves on small counts
- A drop of 0.03 is less than one ticket, so the regression bar is **tickets**: at most 1 more missed P0 than production, paired on the same rows
- **McNemar's test** on the pair says whether the tickets one model alone caught could be chance; it is printed as evidence, not used as the bar

A candidate goes anywhere only by passing the gate

Bar	Value	What it protects
P0 recall, absolute	at least 0.85	Northwind's SLA, whatever the last model did
ECE, absolute	at most 0.12	the threshold means something
P0 recall per language, gate set	at least 0.70 once a language has 10 P0 tickets, or a written waiver	German customers are customers too
Macro-F1 against production	drop of at most 0.02	recall not bought by wrecking three classes
Missed P0 tickets against production	at most 1 more, paired	no quiet regression on the commercial number
Brier against production	increase of at most 0.01	probabilities not getting worse
Same test split	identical data hash	two numbers that mean the same thing

Three fixes, cheapest first, and only calibration makes the threshold work

Fix	Threshold	Macro-F1	P0 recall	P0 precision	The gate says
none	0.50	0.688	0.581, 18/31	0.900	below the bar
class weights	0.50	0.717	0.774, 24/31	0.828	below the bar
a swept threshold, uncalibrated	0.52, target missed	0.717	0.774, 24/31	0.828	below the bar
calibration, then the sweep	0.07	0.681	0.903, 28/31	0.538	PROMOTED

Never on the test split. Validation is split in two: 399 rows fit the calibration, 396 choose the threshold. The test split is the number you report; choosing on it makes that number a lie.

Calibration made two numbers worse and the decision possible

14 of 15

P0 tickets on the threshold half at 0.07 after calibration.
On the uncalibrated scores no threshold reached 0.90

- Brier (P0) 0.0115 to 0.0144, ECE 0.058 to 0.086: slightly worse, and the report says so
- The price of 28 of 31: 21 P1 tickets flagged P0, where the uncalibrated model flagged 3
- German P0 recall on the gate set rises from 0 of 24 to 4 of 24, still far below the bar, and waived in writing

A run is not a promotion, and a precision floor is somebody else's number

weights	calibrated	threshold	macro-F1	P0 recall	P0 precision	
none	no	0.50	0.688	0.581	0.900	
balanced	no	0.52	0.717	0.774	0.828	
balanced	yes	0.07	0.681	0.903	0.538	latest
balanced	yes, floor 0.5	0.33	0.732	0.774	0.923	

The floor row has the best macro-F1 and the fewest false alarms, and it misses four more emergencies. The gate refuses it; whether Northwind wants that trade is the on-call lead's decision.

The pipeline runs your code, cut into four plain steps

Step	Writes
data check	the profile; exit 1 on a blocking finding
train	the versioned candidate, the run record, the card
evaluate	the gate's decision in <code>gate.json</code> , with <code>champion_source</code>
register	the candidate in your registry, only if the gate passed

`make pipeline-submit` ships your `nw/` as a source bundle; the step log shows `nw source <sha> (commit <sha>)`. Rebuild the pipelines image only when dependencies change.

One step code, two definitions

Local and Google Cloud: the same compiled Kubeflow YAML, on the local runner and on Pipelines.

AWS: a SageMaker definition, with a ConditionStep on the gate's `passed_int` and a FailStep that carries its reasons.

Azure: Azure Machine Learning pipelines on serverless compute.

The card says what the report did not

4 of 24

German PO tickets caught on the gate set, against 40 of 53 English ones. The test split alone holds two German POs: too few to judge

`MODEL_CARD.md` is generated from the artifact, so it cannot go stale:

- intended use; the model, threshold and targets; the data hash
- validation 795 rows: 399 calibrate, 396 choose the threshold
- the test table, then per language: de 39 rows, macro-F1 0.536; en 755 rows, 0.688
- the gate-set table with the German line WAIVED, and why

`promotions.jsonl` beside it: every gate decision, who made it, and `"forced": true` when someone overrode one.

A registry version is an artifact, its measurements and a stage

Stage	Who moves it	What it means
candidate	the pipeline's register step	cleared the gate; nobody has read it yet
approved	a reviewer, <code>set_stage(..., Stage.APPROVED, reason)</code>	served on your tenant's endpoint
live	the capstone's promotion drill, with a canary	served to customers on the live target
retired	the next <code>live</code>	kept for rollback and audit

The artifact is immutable. The stage is the only thing that changes, and it changes by a person, with a sentence. Approving a version returns the previously approved one to `candidate`.

Approval is the same call on four platforms

AWS

The package turns `Approved` ; the state change event reaches `northwind-deploy-on-approval` , which creates your Serverless Inference endpoint. Solo mode deploys it directly.

Google Cloud

The alias moves to `approved` with your reason in `stages.jsonl` .
`endpoints.deploy` rewrites `NW_MODEL_URI` on your Cloud Run service: a new revision reading the approved artifact.

Azure

The `stage` tag on the Azure Machine Learning model moves to `approved` , reason in `stages.jsonl` .
`endpoints.deploy` puts it on your managed online endpoint.

Local

The MLflow alias and the `stage_reason` tag.
`deploy(live=True)` restarts `serving-stable` on the registry's artifact; the proxy on :8005 reads `stable 100, canary 0` .

`approved` is not `live` . What you deployed is your tenant's endpoint, which nobody but you calls. Customers see a model only through the capstone's canary.

A model that fails to load leaves the process alive and not ready

Endpoint	Who calls it
<code>/healthz</code>	the liveness probe; failing it restarts
<code>/readyz</code>	the load balancer; failing it stops traffic
<code>/trriage</code>	the agent, the capstone router
<code>/metrics</code> , <code>/version</code> , <code>/drift</code>	dashboards, "which model", "which world"

On the platform the service reads `NW_MODEL_URI` from the registry, not `latest`.

Load fails

Process stays up, `ready = False`, one log line with the path or URI and the error.

`/healthz` 200, `/readyz` and `/trriage` 503.

Crashing loops the restart and scrolls the one useful line off the screen.

Two hundred one-line tickets, and the service says the world has changed

8.87

text-length PSI after 200 one-line tickets. Priority 3.74; the P0 share 0.06 times what the model predicted on validation. Level: alert

- **PSI** compares two histograms: under 0.1 stable, 0.1 to 0.2 watch, over 0.2 alert
- A small window crosses 0.2 **by chance**: at 50 requests about one stationary snapshot in four did. So the monitor waits for 200, and raises its bar to the PSI a stationary window of that size stays under 99 times in 100
- Past the bar: one `drift_alert` log line, the alarm on every track is built on it

The platform watches from outside, and only a person decides to retrain

Track	The watcher	What fires
AWS	the service's <code>drift_alert</code> metric filter; captured traffic backtested	a CloudWatch alarm, SNS
Google Cloud	Model Monitoring, created by a script, not by Terraform	a log-based metric, an alert policy
Azure	model monitoring is not deployed	a log alert on <code>drift_alert</code> , an action group
Local	Evidently, a report every 15 minutes on :8030	nothing: Local has no alerting

SageMaker Model Monitor is no longer open to new customers, so the AWS track does not rely on it.

On twenty emergencies the shadow looks better; on the whole split it costs 26 false alarms

A candidate trained for recall 0.95 picks threshold 0.03. Twenty real P0 tickets through the shadow slot:

`nw_triage_shadow_total` agree 19, disagree 1: the shadow caught the one the served model sent to P1.

backtest, 794 tickets	Macro-F1	P0 recall	P0 precision
served, 0.07	0.681	28 of 31	0.538
candidate, 0.03	0.635	29 of 31	0.372

Agreement 0.967; every disagreement is the candidate saying P0: 17 from P1, 7 from P2, 2 from P3. `GATE FAILED`: macro-F1 drops more than 0.02.

The retraining job exists, is paused, and never promotes

Track	The weekly job
AWS	EventBridge rule <code>northwind-<handle>-retrain-triage</code> , disabled
Google Cloud	Cloud Scheduler job, paused
Azure	Azure Machine Learning schedule, disabled
Local	<code>make pipeline-submit</code> by hand

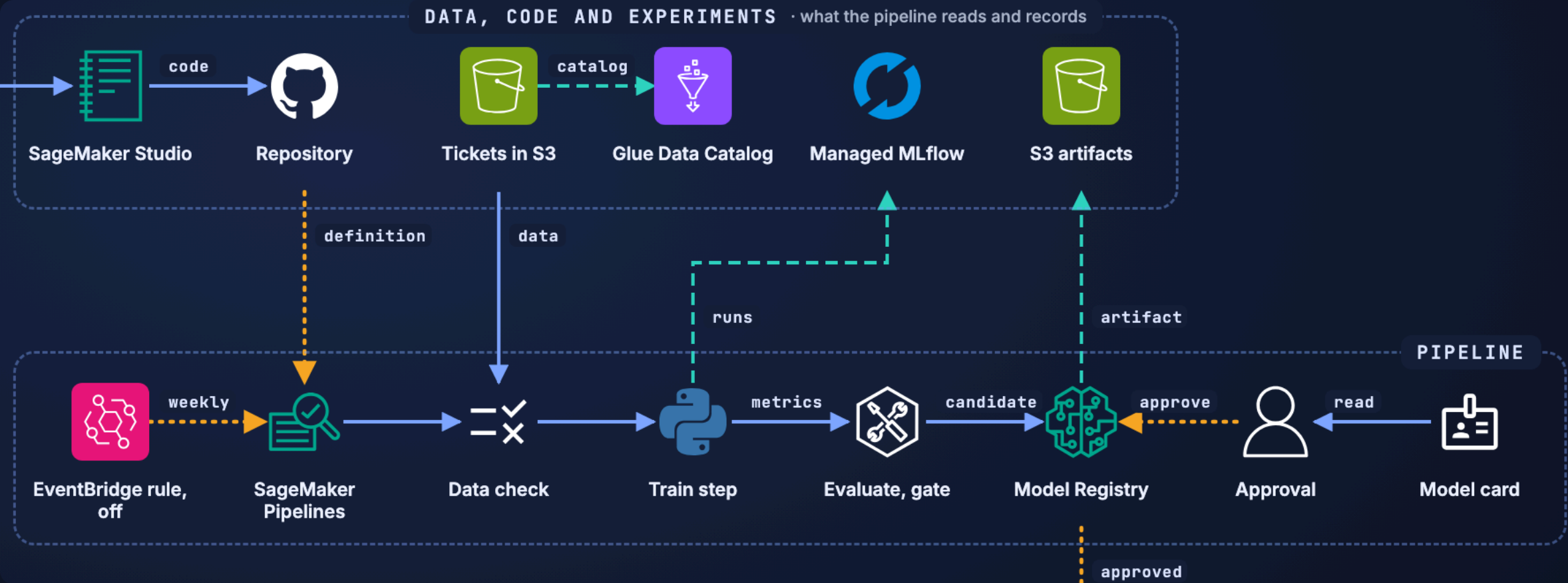
Every gate bar is a pipeline parameter: `ARGS="--min-p0-recall 0.9"` still registers at 0.903; 0.95 is refused and nothing is registered.

Two places define green

`retrain-triage.yml` runs weekly, on demand and on pull requests, gates against the committed summary, and registers nothing. With the same data and no signal it skips.

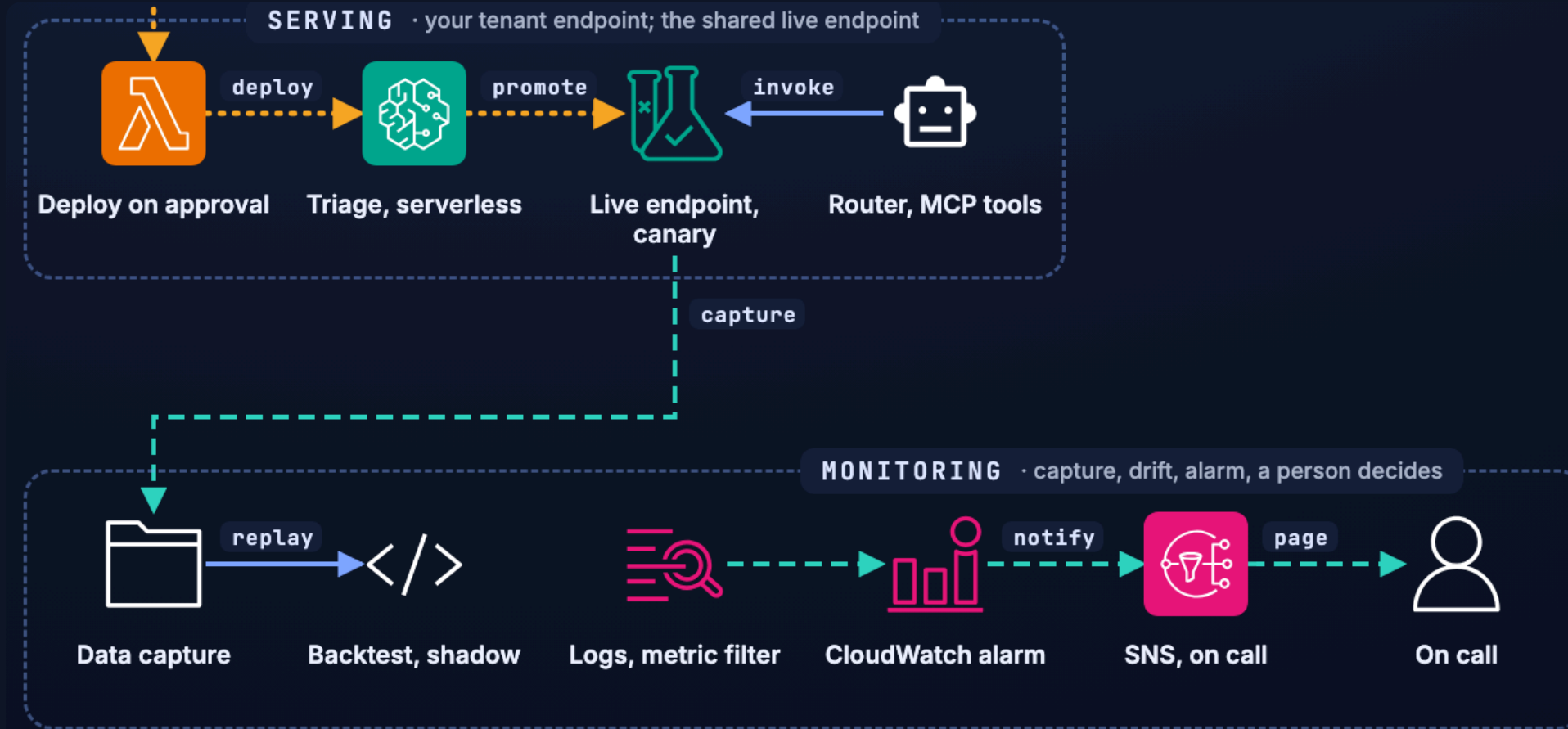
The registry approval is the other. Both read the same gate.

The pipeline is the only path to the registry (AWS)



Data, code and experiments feed SageMaker Pipelines: data check, train, evaluate and gate, then a candidate in the Model Registry that waits for a person who has read the card. The weekly EventBridge rule ships off.

Approval deploys; capture and one log line close the loop (AWS)



An approval deploys your serverless triage endpoint; the capstone promotes to the live endpoint behind a canary. Captured traffic is backtested, and the drift_alert metric filter feeds a CloudWatch alarm that pages a person.

Acceptance criteria

1. The data passes its contract; the profile ships in the artifact
2. P0 recall at or above the bar by the threshold rule; the gate passed
3. The pipeline registered one candidate: 0.681 macro-F1, 0.903 P0 recall, 0.538 precision, 0.086 ECE
4. You approved it with a reason and your endpoint serves that version
5. The service: probes, metrics, correlation ID, drift, shadow

59 passed

what `make session02` prints in the solution when you are done

Questions worth asking

- Northwind's Enterprise P0 SLA is 15 minutes. What recall does that imply, and what precision can on-call absorb?
- The German line is waived until December. Who should sign that, and what would lift it?
- Your approval reason is on the registry for good. What would you want it to say in six months?
- What would change if the model became a transformer? What would stay?

RESERVE

Definitions, evidence and the other tracks

Supervised and unsupervised learning, where small models run today, then the MLOps figures for Google Cloud, Azure and Local.

Supervised learning predicts a label; unsupervised learning finds structure

Supervised

Learns to predict explicit, often human-given labels.

At Northwind: priority, learned from 7,950 tickets someone already labelled. That is Project 1.

Also: fraud, churn, credit risk, demand

Unsupervised

Learns patterns in data nobody labelled, for example clusters.

At Northwind: grouping this week's tickets to find a topic no tag covers yet.

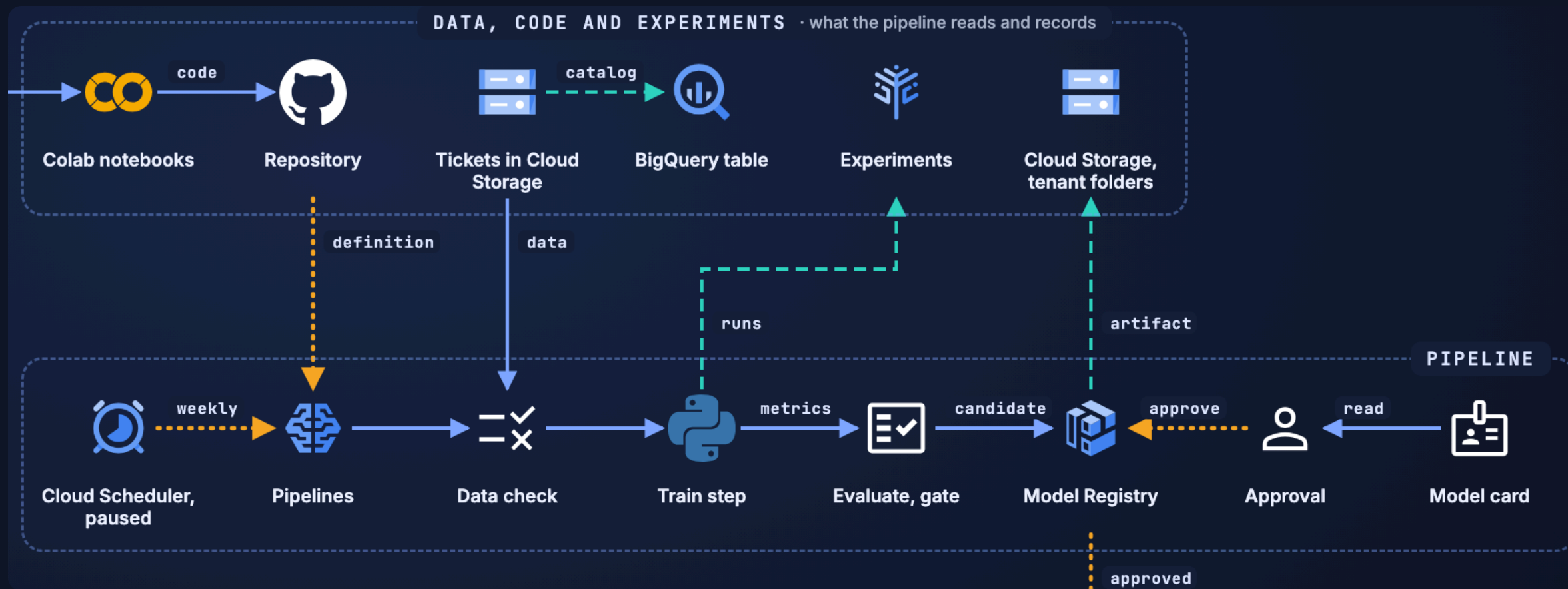
Also: segmentation, anomaly detection

"Classical" means the models before deep learning: linear models, trees, gradient boosting. They are still the default for tabular data and narrow text tasks.

Small, purpose-built models still run many decisions that need milliseconds and a reason

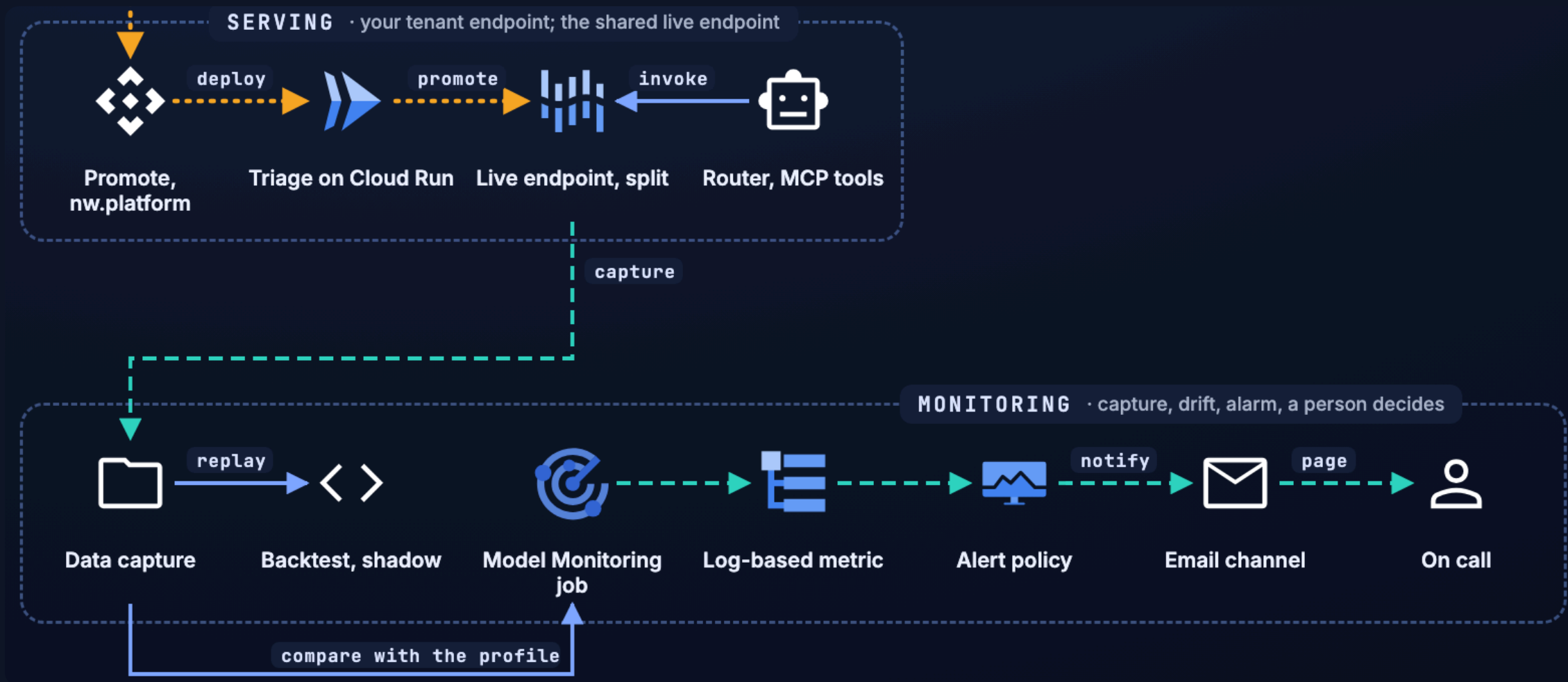
Where	What the evidence says
Tabular competitions, 2025	XGBoost and LightGBM each in 14 winning solutions, CatBoost in 8
Uber's platform, 2024	over 5,000 production models, 10 million predictions a second at peak; "in several cases, XGBoost outperforms DL"
Stripe Radar	each payment scored in under 100 ms on over 1,000 signals; 0.1 percent wrongly blocked
Credit scoring, EU	high-risk under the AI Act; fraud detection is explicitly excluded
Benchmarks, 2025 and 2026	trees stay competitive on tabular data; tabular foundation models lead on small data; no family wins everywhere

The pipeline, Google Cloud track



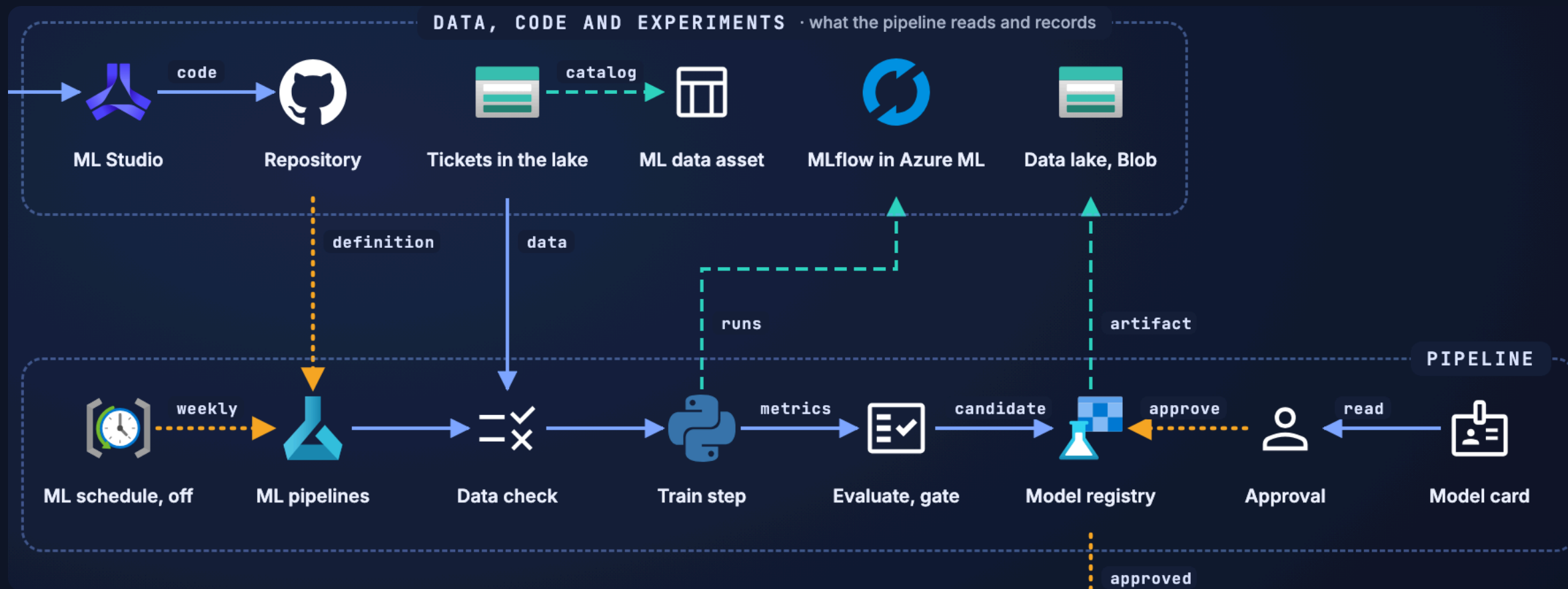
Tickets in Cloud Storage and BigQuery feed Pipelines on the Agent Platform (formerly Vertex AI), running the same Kubeflow YAML as Local; the Model Registry holds the candidate under an alias until a person approves. Cloud Scheduler ships paused.

Serving and monitoring, Google Cloud track



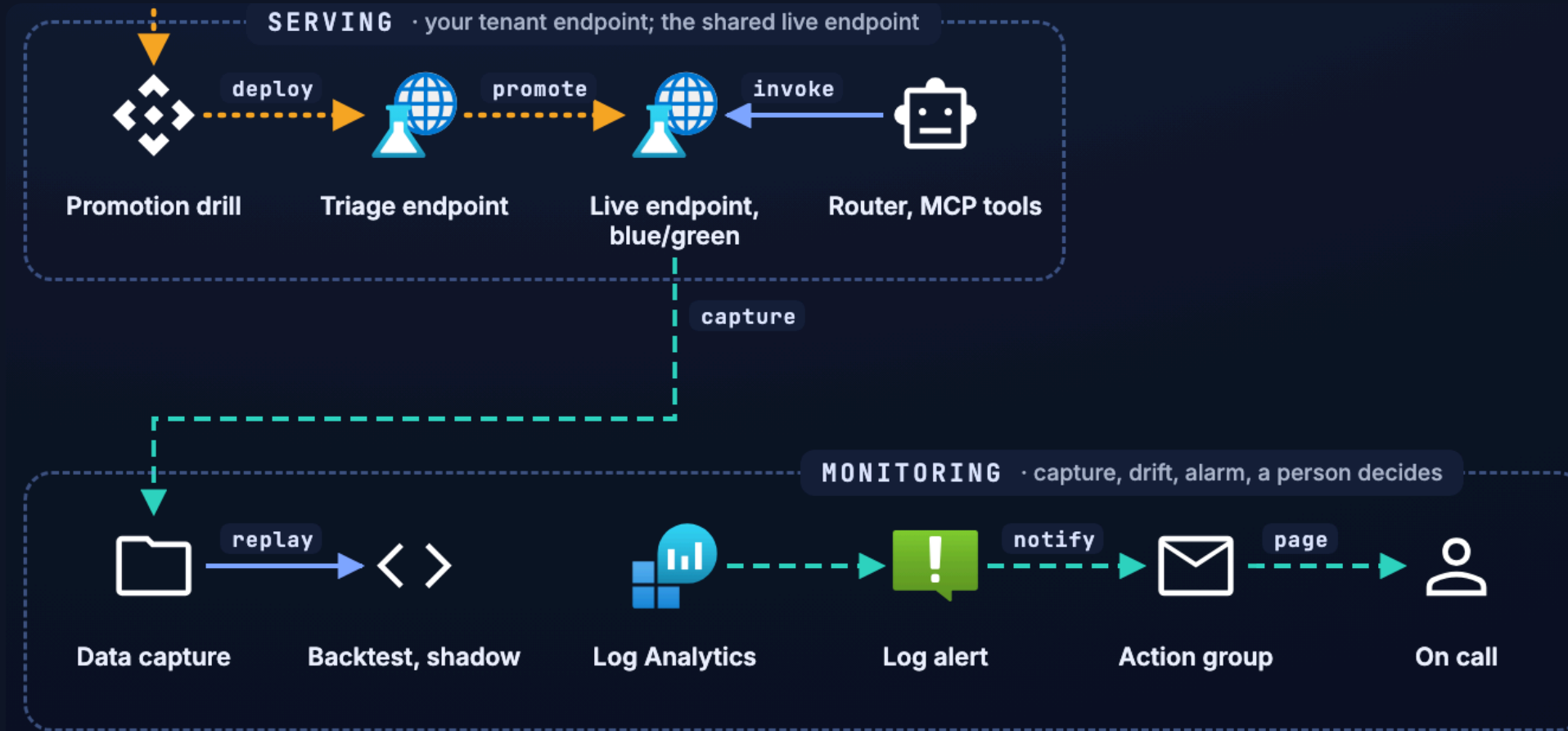
Your tenant's triage runs on Cloud Run from the approved artifact; the live model sits on an endpoint with a traffic split. The Model Monitoring job compares captures with the profile; a log-based metric on drift_alert raises an alert policy that emails on-call.

The pipeline, Azure track



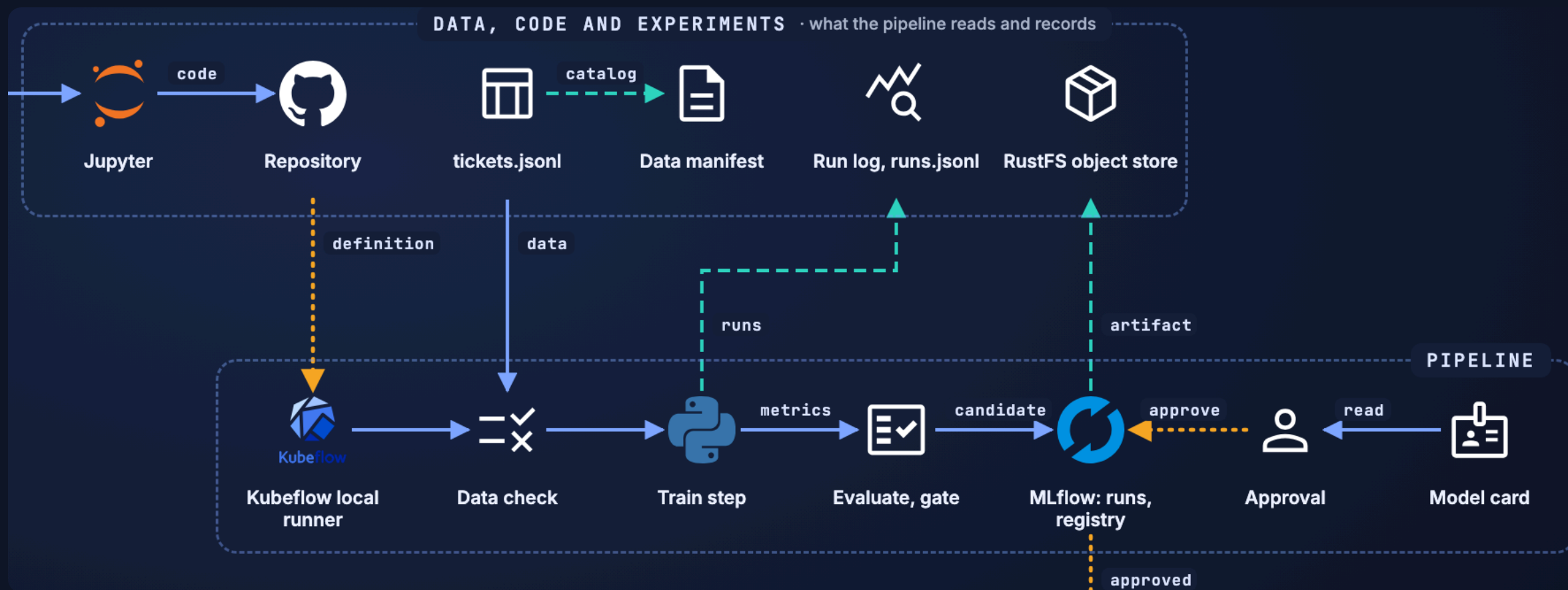
Tickets in the lake and an ML data asset feed Azure Machine Learning pipelines on serverless compute; the model registry carries the stage as a tag; a person approves after reading the card. The ML schedule ships off.

Serving and monitoring, Azure track



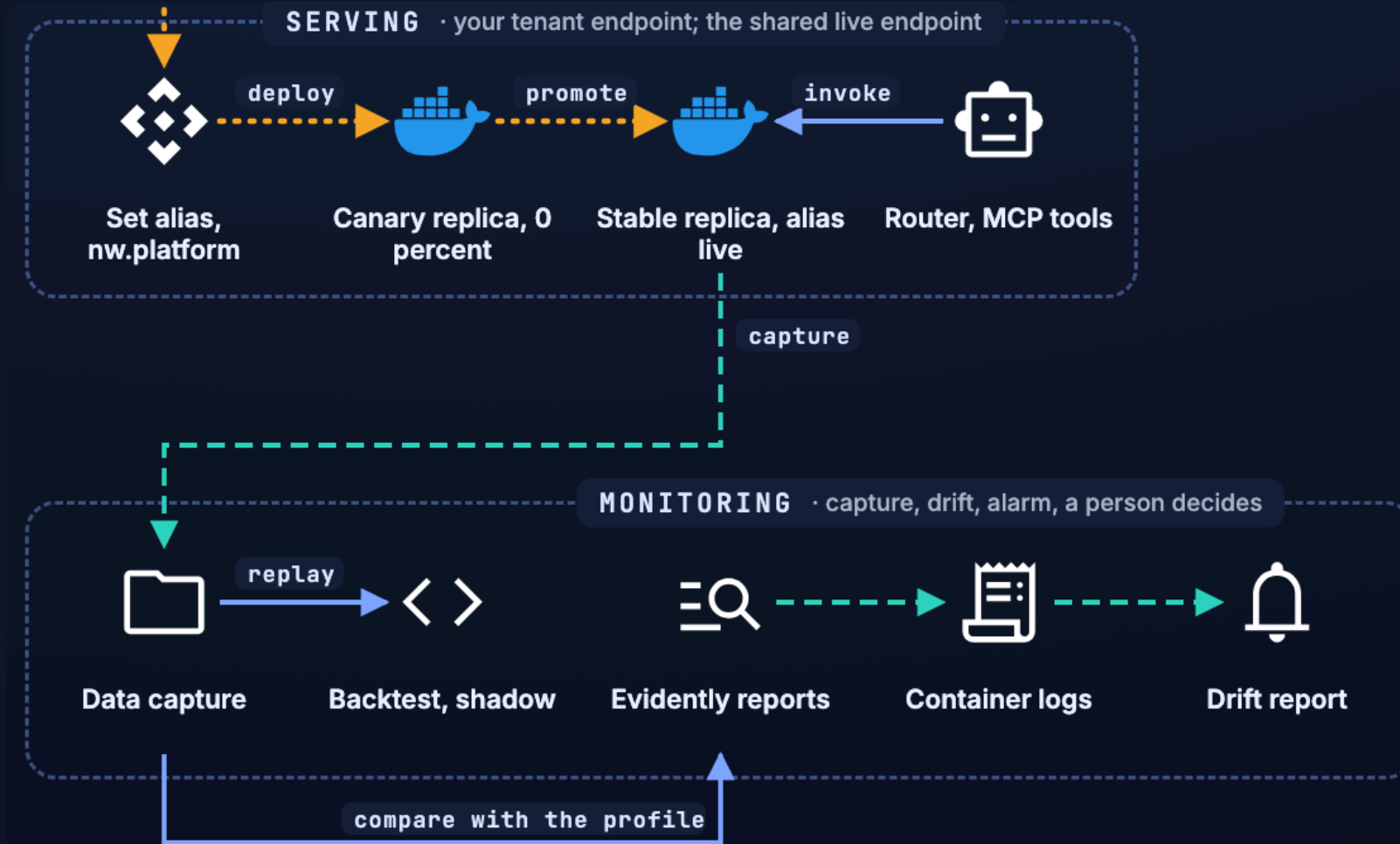
An approval deploys your triage managed online endpoint; the live endpoint runs blue and green deployments. Captures go to Log Analytics, and a log alert on drift_alert notifies an action group and on-call.

The pipeline, Local track



tickets.jsonl and its data manifest feed the Kubeflow local runner; MLflow holds the runs and the registry, with aliases; RustFS holds the artifacts. The approval is the same call as on the clouds.

Serving and monitoring, Local track



Setting the alias restarts the stable replica on the registry's artifact; the canary replica sits at zero percent behind the weighted nginx proxy on :8005. Evidently writes a drift report every 15 minutes from the capture files.